

AI ASSISTED CODING – LAB ASSIGNMENT

Assignment 12.1

NAME:K.SAI VARDHAN REDDY

ROLL NO:2503A51266

Task 1 – Stack Implementation with AI Assistance

AI Prompt: Create a Python Stack class with push(element), pop(), peek(), and is_empty() methods. Include proper stack-underflow error handling and clear docstrings for every method.

Solution explanation: A stack follows the LIFO (Last In, First Out) principle. The most recently pushed element is removed first. The implementation uses a Python list as the internal storage.

- push(element): adds an element to the top.
- pop(): removes and returns the top element.
- peek(): returns the top element without removing it.
- is_empty(): checks whether the stack contains no elements.
- Underflow is handled by raising an IndexError when pop or peek is attempted on an empty stack.

CODE:

```
stack.py
1 class Stack:
2     """A stack implementation using a Python list."""
3
4     def __init__(self):
5         self.items = []
6
7     def push(self, element):
8         """Add an element to the top of the stack."""
9         self.items.append(element)
10
11    def pop(self):
12        """Remove and return the top element."""
13        if self.is_empty():
14            raise IndexError("Stack underflow: stack is empty")
15        return self.items.pop()
16
17    def peek(self):
18        """Return the top element without removing it."""
19        if self.is_empty():
20            raise IndexError("Stack underflow: stack is empty")
21        return self.items[-1]
22
23    def is_empty(self):
24        """Return True if the stack is empty."""
25        return len(self.items) == 0
26
27
28 stack = Stack()
29 stack.push(10)
30 stack.push(20)
31 print("Top:", stack.peek())
32 print("Popped:", stack.pop())
33 print("Empty:", stack.is_empty())
```

OUTPUT:

```
stack.py

> python stack.py
Top: 20
Popped: 20
Empty: False
```

Task 2 – Linear Search vs Binary Search

AI Prompt: Generate Python implementations of `linear_search(arr, target)` and `binary_search(arr, target)`, with comments/docstrings describing their working principles, time complexity, and space complexity. Test both algorithms on a sorted list.

Solution explanation: Linear Search checks elements sequentially from the beginning until the target is found or the array ends. Binary Search repeatedly divides a sorted search range into two halves and therefore requires sorted input.

- Linear Search time complexity: $O(n)$ in the worst case; $O(1)$ extra space.
- Binary Search time complexity: $O(\log n)$ in the worst case; $O(1)$ extra space for the iterative implementation.
- For a sorted large dataset, Binary Search is generally more efficient

CODE:

```
search.py

search.py
1 def linear_search(arr, target):
2     """Linear search: checks elements one by one.
3     Time:  $O(n)$ , Space:  $O(1)$ .
4     """
5     for i, value in enumerate(arr):
6         if value == target:
7             return i
8     return -1
9
10
11 def binary_search(arr, target):
12     """Binary search on a sorted array.
13     Time:  $O(\log n)$ , Space:  $O(1)$ .
14     """
15     low, high = 0, len(arr) - 1
16
17     while low <= high:
18         mid = (low + high) // 2
19
20         if arr[mid] == target:
21             return mid
22         elif arr[mid] < target:
23             low = mid + 1
24         else:
25             high = mid - 1
26
27     return -1
28
29
30 arr = [5, 10, 15, 20, 25, 30, 35]
31 target = 25
32
33 print("Linear Search:", linear_search(arr, target))
34 print("Binary Search:", binary_search(arr, target))

Python UTF-8 LF
```

OUTPUT:

```
search.py

> python search.py
Linear Search: 4
Binary Search: 4
```

Task 3 – Test Driven Development (TDD): Simple Calculator

AI Prompt: First create unit tests for addition and subtraction. The tests should be executed before the implementation so that failures are observed. Then generate the calculator implementation required to make all tests pass, and run the tests again.

TDD process:

- Step 1 – Write the tests first for addition and subtraction.
- Step 2 – Run the tests before implementation; they fail because the required functions do not yet exist.
- Step 3 – Implement add() and subtract().
- Step 4 – Run the tests again and verify that all tests pass.

CODE:

```
test_calculator.py

test_calculator.py
1  from calculator import add, subtract
2
3
4  def test_addition():
5      assert add(10, 5) == 15
6      assert add(-2, 3) == 1
7
8
9  def test_subtraction():
10     assert subtract(10, 5) == 5
11     assert subtract(3, 8) == -5
12
13
14 if __name__ == "__main__":
15     test_addition()
16     test_subtraction()
17     print("All tests passed!")
```

Python UTF-8 LF

```
calculator.py

calculator.py
1  def add(a, b):
2      """Return the sum of two numbers."""
3      return a + b
4
5
6  def subtract(a, b):
7      """Return the difference of two numbers."""
8      return a - b
```

Python UTF-8 LF

OUTPUT:

```
test_calculator.py  
  
> python test_calculator.py  
All tests passed!
```

Task 4 – Queue Implementation with AI Assistance

AI Prompt: Create a Python Queue class with enqueue(element), dequeue(), front(), and is_empty() methods. Handle queue overflow and underflow conditions and include appropriate docstrings.

Solution explanation: A queue follows the FIFO (First In, First Out) principle. The first element inserted into the queue is the first element removed.

- enqueue(element): inserts an element at the rear.
- dequeue(): removes and returns the front element.
- front(): returns the front element without removing it.
- is_empty(): checks whether the queue is empty.
- Overflow is handled when the configured maximum size is reached.
- Underflow is handled when dequeue or front is requested on an empty queue.

CODE:

```
queue.py  
1 class Queue:  
2     """A queue implementation using a Python list."""  
3  
4     def __init__(self, max_size=5):  
5         self.items = []  
6         self.max_size = max_size  
7  
8     def enqueue(self, element):  
9         """Add an element to the rear of the queue."""  
10        if len(self.items) >= self.max_size:  
11            raise OverflowError("Queue overflow: queue is full")  
12        self.items.append(element)  
13  
14        def dequeue(self):  
15            """Remove and return the front element."""  
16            if self.is_empty():  
17                raise IndexError("Queue underflow: queue is empty")  
18            return self.items.pop(0)  
19  
20        def front(self):  
21            """Return the front element without removing it."""  
22            if self.is_empty():  
23                raise IndexError("Queue underflow: queue is empty")  
24            return self.items[0]  
25  
26        def is_empty(self):  
27            """Return True if the queue is empty."""  
28            return len(self.items) == 0  
29  
30  
31 queue = Queue(max_size=3)  
32 queue.enqueue(10)  
33 queue.enqueue(20)  
34 print("Front:", queue.front())  
35 print("Dequeued:", queue.dequeue())  
36 print("Empty:", queue.is_empty())
```

Python UTF-8 LF

OUTPUT:

```
queue.py

> python queue.py
Front: 10
Dequeued: 10
Empty: False
```

Task 5 – Bubble Sort vs Selection Sort

AI Prompt: Generate Python implementations of `bubble_sort(arr)` and `selection_sort(arr)`. Include comments explaining the major steps and docstrings stating time and space complexity. Test both algorithms on the same input.

Solution explanation: Both algorithms sort the input in ascending order.

- Bubble Sort repeatedly compares adjacent elements and swaps them when they are in the wrong order.
- Selection Sort repeatedly finds the smallest remaining element and places it at the correct position.
- Bubble Sort worst/average time: $O(n^2)$; best time can be $O(n)$ with the early-stop optimization.
- Selection Sort time: $O(n^2)$ in best, average, and worst cases.
- Both use $O(1)$ auxiliary space in the in-place sorting logic; the submitted functions first copy the input so the original list remains unchanged.

```
sorting.py

1 def bubble_sort(arr):
2     """Bubble Sort.
3     Time:  $O(n^2)$  worst/average,  $O(n)$  best.
4     Space:  $O(1)$ .
5     """
6     a = arr.copy()
7
8     for i in range(len(a)):
9         swapped = False
10        for j in range(0, len(a) - i - 1):
11            if a[j] > a[j + 1]:
12                a[j], a[j + 1] = a[j + 1], a[j]
13                swapped = True
14        if not swapped:
15            break
16
17    return a
18
19
20 def selection_sort(arr):
21     """Selection Sort.
22     Time:  $O(n^2)$  in all cases.
23     Space:  $O(1)$ .
24     """
25     a = arr.copy()
26
27     for i in range(len(a)):
28         min_index = i
29         for j in range(i + 1, len(a)):
30             if a[j] < a[min_index]:
31                 min_index = j
32
33         a[i], a[min_index] = a[min_index], a[i]
34
35    return a
36
37
38 data = [64, 25, 12, 22, 11]
39
40 print("Bubble Sort:", bubble_sort(data))
41 print("Selection Sort:", selection_sort(data))
```

Python UTF-8 LF

OUTPUT:

```
● ● ● sorting.py  
  
> python sorting.py  
Bubble Sort: [11, 12, 22, 25, 64]  
Selection Sort: [11, 12, 22, 25, 64]
```